

Factored AI & Data Hackathon 2026: Strategy

This is based on the *Problem Statement* and the *Kickoff* slides (2026-09-25).

The challenge isn't about marketing or promotions. The task is to build an **AI customer-service system for one banking workflow** and prove with measurements that it's safe and works better than a simple baseline. The judges score depth, rigor and honesty, not how many features you have.

What the judges want

They ask for	What it means in practice
One focused workflow: account/payment questions, card support, transaction disputes, or credit eligibility	Pick one and go deep. Extra workflows earn no bonus.
Spanish and Portuguese	Every demo and test must work in both languages.
3 paths	One the AI resolves itself, one ambiguous or unsupported request (it asks or declines), and one handed to a human with a structured summary.
Rules enforced in code, not in the prompt	Tools check the logged-in session themselves. The LLM can never see another customer's data, whatever it's asked.
Authentication	A test login session. A customer number alone doesn't count as proof of identity.
Baseline vs your system on held-out cases	Report safe automated resolution, containment, escalation quality, unsafe outcomes, p50/p95 latency and cost, with sample sizes.
Attack and failure tests	Prompt injection, attempts to access another customer, expired sessions, tool failures, missing data, mixed languages.
Data engineering	A repeatable pipeline with schema contracts, quality checks, lineage and a policy for loading new data.
At least one learned component evaluated against a baseline	No leakage, a proper split, and justified metrics and thresholds.
Ready for production	Tracing, bounded retries, safe fallback, reproducible setup, and an honest list of what's missing.

What to submit: a public GitHub repo named `factored-hackathon-2026-[team]`, a deployed link, 4–6 slides and a demo video. Send them to hackathon.admin@factored.ai. The first prize is US\$6,000 plus an interview at Factored.

Recommendation: transaction disputes

Choose **transaction dispute intake**, for example "there's a charge on my card I don't recognise". The data supports it better than any other workflow:

- **"Transaccional" is the top contact reason:** 35% of calls on one sampled day (2025-03-15), with "Queja" (complaint) second at 20%. That's the "why this problem matters" slide. It still needs to be confirmed across all three years.
- `complaints` already has category **Transactions**, subcategory **"Cargo no reconocido"** (unrecognised charge), the amount claimed, whether the SLA was missed, the compensation, and the resolution.
- `transactions` lets the system *check the facts* of the claim instead of trusting it: that the charge exists, belongs to the logged-in customer, and has the stated amount, merchant, date and status. It also has a strong risk signal: in H1 2025, `fraud_score` has a median of 48.6 on fraud-flagged transactions vs 15.0 on the rest. That score can decide routing, but it is **not proof of fraud**. Confirmed fraud is rare (715 of 722,568 transactions in H1 2025, about 0.1%), so use the score as a plain rule threshold rather than training a fraud model.
- The need for a human is obvious and easy to defend: high amounts, suspected fraud, repeat complainers, disputes that are too old.

Card support is a fine second choice. **Credit eligibility** is the riskiest: it needs a separate rules service, and the rules say the LLM must never approve credit on its own.

How to build it



- **Pipeline:** S3 → daily partitions → Parquet/DuckDB, with schema checks (for example pandera) and a data-quality report. The problems already found belong in the report: broken branch links, the BOM at the start of the CSVs, "México" vs "Mexico", Mexican products in USD, template transcripts. The daily partitions also make incremental loading easy to demonstrate.
- **Learned component:** a model that predicts whether a dispute needs a human (or its priority). Train it on complaints from 2023–2025 and test it on 2026, so no future data leaks into training. Compare it with a plain-rules baseline.
- **Evaluation:** about 150–300 scripted conversations in ES and PT, each labelled with the correct outcome, including attack cases. Run the baseline (for example, the LLM without the tools or rules) and the proposed system on the same set, several times each. If an LLM judges the answers, check a sample of its verdicts by hand.

Limitations to report

- **There's no Portuguese in the data.** `detected_language` is always `es`, so all Portuguese test cases will be written by the team. Say that openly.

- Transcripts are templates (44 different texts across 113 calls on the sampled day) and `detected_intents` is always `consulta_general`. You can't train an intent model on them, so the team needs to create labelled examples.
- **Complaints can't be linked to specific transactions.** `complaints` has no transaction ID, and in H1 2025 none of 480 "Transactions" complaints with a product and amount matched a real transaction of that product and amount. So there is no historical label for whether a dispute was legitimate. Build the test set by picking real transactions (fraud-flagged and normal) and writing dispute conversations about them, labelled from those transactions.
- The data is synthetic, so any savings you calculate are *projections*, not measured results.

Before anything else

- **The repo will be public.** Keep `.env`, `data/` and `.venv/` in `.gitignore`, so AWS keys and data never get committed, and name the repo `factored-hackathon-2026-[team]`.
- **Check the deadline** on the timeline slide of the kickoff deck. If the sprint is 10 days from the Sept 25 kickoff, the deadline is around Oct 5.
- **Rough 10-day plan:**
 - Days 1–2: data analysis and choosing the workflow.
 - Days 3–5: pipeline, tools, rules engine and agent.
 - Days 6–7: learned component and test set.
 - Day 8: attack and failure tests.
 - Day 9: deploy and write docs.
 - Day 10: slides and video.
- **Submit something, no matter what.** The kickoff slides say this explicitly.